FSG — QF Substrate Requirements

What Fractal Sovereign Guard needs from a host infrastructure layer

Technical Brief | Draft 1.0 | March 2026

Purpose of this document

Fractal Sovereign Guard (FSG) is a governance architecture for multi-agent AI systems. It manages agent lifecycle, integrity validation, and coordination across a swarm of AI instances. FSG needs a host infrastructure layer to perform two specific jobs that cannot live inside any single agent:

- Immutable, replayable state snapshots — a tamper-evident record of agent lifecycle events that no individual agent can modify or delete.
- Cross-agent coordination — a shared substrate that allows agents to coordinate without any single agent owning or controlling the coordination layer.

QF Network is a candidate host for both of these functions. This document specifies what FSG needs from that host, what it will write and read, and what a minimal integration looks like. It does not require QF to understand the full FSG architecture — only to provide the primitives listed here.

What FSG writes to the host

FSG generates four categories of write operations. All writes are append-only. FSG never needs to modify or delete a prior write — the immutability is a feature, not a constraint.

Write type	Trigger	Approximate frequency	Size estimate
Lifecycle event	Agent spawned, dissolved, promoted, quarantined	Per lifecycle transition	< 1 KB
Integrity snapshot	Periodic checkpoint of agent integrity vector	Every N decision windows	< 2 KB
Validation commitment	Hash of a validated policy or evidence bundle	Per validated action batch	< 256 bytes
Replay fragment digest	Approved trajectory summaries for reinstantiation	On prestige reincarnation	< 4 KB

FSG does not require the host to interpret these writes. The host is a commitment surface, not a validator. All semantic interpretation stays in the FSG layer.

What FSG reads from the host

FSG reads from the host in three situations:

- Reinstantiation — when an agent is restarted, FSG retrieves its last approved integrity snapshot and any authorized replay fragments.
- Audit — when a governance query requires reconstructing an agent's history, FSG reads the append-only lifecycle event log.
- Pulse consensus — when aggregating signals across agents, FSG reads recent governance signals written by peer agents to compute a collective pulse.

Read latency requirements are not strict. FSG can tolerate eventual consistency for most reads. The exception is reinstantiation, where the snapshot must be retrievable within a reasonable window to avoid extended downtime.

What FSG needs the host to guarantee

These are the hard requirements. Everything else is implementation detail.

Requirement	Why FSG needs it	Hard or soft
Append-only writes	FSG's audit and replay depends on records that cannot be retroactively altered by any agent or operator.	Hard
Tamper evidence	Any modification to a committed record must be detectable. Hash-chaining or Merkle-style commitments satisfy this.	Hard
No single-agent ownership	The coordination layer must be outside any agent's control surface. If one agent can write or delete records belonging to another, the governance model breaks.	Hard
Persistent storage across agent lifecycle	Snapshots must survive agent dissolution and be retrievable on reinstantiation. Agent memory is ephemeral by design.	Hard
Permissioned write access	Only authorized FSG components can write	Hard

Requirement	Why FSG needs it	Hard or soft
	lifecycle events and snapshots. Arbitrary agents cannot pollute the ledger.	
Off-chain compute compatibility	All FSG validation, proposal generation, and training logic runs off-chain. The host stores commitments only, not executable logic.	Hard
Queryable by record type and agent ID	FSG needs to retrieve records for a specific agent without scanning the full ledger.	Soft — indexing can be handled off-chain if needed

What lives on-chain vs off-chain

This split is important. FSG does not ask QF to become a validator or a governance engine. The chain stores commitments. The intelligence stays off-chain.

Component	Lives where	Notes
Lifecycle event log	On-chain	Append-only, tamper-evident, permanent
Integrity snapshots	On-chain	Periodic checkpoints, retrievable on restart
Validation commitments	On-chain	Hashes only — raw evidence stays off-chain
Replay fragment digests	On-chain	Digests only — full fragments stored off-chain
Proposal generation	Off-chain	Fast inference, not suitable for chain execution
Deterministic validation	Off-chain	Truth-gating logic runs in FSG validator layer
Training island operation	Off-chain	Isolated agent training environment
Pulse consensus computation	Off-chain	Aggregation computed off-chain, result committed on-chain
Supervisor overrides	Off-chain	Human-in-the-loop controls stay outside chain

Minimal integration path

FSG does not need full integration on day one. A staged path reduces risk for both teams.

Stage 1 — Single agent, off-chain prototype

Build the FSG validator stack, karmic ledger, and rebirth loop entirely off-chain. No chain integration yet. Validate that the lifecycle mechanics work in isolation.

Stage 2 — Commitment layer

Anchor lifecycle events and integrity snapshot digests to QF. This is the first real integration point. FSG writes; QF stores; FSG reads back on restart. Validate tamper evidence and retrieval reliability.

Stage 3 — Multi-agent pulse pilot

Run several off-chain agents writing governance signals to QF. Compute pulse consensus off-chain using on-chain signal records. This tests the cross-agent coordination function at small scale.

Stage 4 — Hardening and scale

Add permissioned write controls, audit replay drills, and fault injection under testnet conditions. Expand agent count. Introduce narrow-scope production workflows only after the single-agent rebirth loop is stable at scale.

What FSG does not need from QF

Equally important as the requirements above:

- FSG does not need QF to run validation logic, truth-gating, or integrity scoring on-chain.
- FSG does not need QF to understand agent semantics or interpret governance events.
- FSG does not need QF to enforce agent behavior — enforcement is FSG's job.
- FSG does not need smart contract execution for the core use case. Append-only storage with tamper evidence is the primary primitive.

The right framing is: QF is the memory substrate that no agent owns. FSG is the governance layer that decides what gets written and what it means.

Next steps

The immediate questions for the QF team are:

- Does QF's current architecture support append-only, tamper-evident writes with permissioned access?
- What is the expected write throughput and storage cost at the frequencies described above?

- Is there an existing off-chain indexing layer, or would FSG need to build one for record retrieval by agent ID?
- What does a minimal testnet integration look like for Stage 2 above?

FSG is being developed in parallel. The validator stack, rebirth loop, and integrity scoring do not depend on chain integration to proceed. The integration can be staged cleanly once QF confirms the substrate primitives are available.

Document prepared by: Singularity (@showthewayup) | FSG v1.3 | March 2026